Documento de Diseño Arquitectónico: Sistema de Archivos Distribuido de Alta Disponibilidad (DFSha)

Sebastian Quiceno Loazia

1. Resumen del Alcance y la Arquitectura

Para desarrollar el **Sistema de Archivos Distribuido de Alta Disponibilidad** (Distributed File System - High Availability), se va a priorizar una solucion con alta resistencia a problemas comunes (puntos únicos de fallo, cuellos de botella en ancho de banda y capacidad finita de disco), proporcionando un entorno resistente a cargas pesadas, escalable horizontalmente y transparente para el usuario final.

El sistema usa una **arquitectura híbrida de comunicaciones** combinando:

1. **RPC (Remote Procedure Call)**: Ideal para operaciones sincrónicas de baja latencia donde el cliente o los componentes internos demandan transferencias inmediatas de datos e interactividad en tiempo real.
2. **MOM con Colas (Message-Oriented Middleware QUEUEs)**: Ideal para el desacoplamiento de cargas pesadas, la persistencia de peticiones en colas seguras, la absorción de ráfagas de tráfico (traffic throttling) y la coordinación asíncrona entre nodos.

A través de la separación estricta entre la **capa de control/metadatos** y la **capa de almacenamiento físico de datos**, el sistema garantiza alta disponibilidad, tolerancia a fallos ante la desconexión imprevista de nodos y un balanceo de carga adaptativo que procura evitar que los servidores se saturen.

1.2. Objetivos Específicos

1. **Desacoplar el plano de control del plano de datos:** Separar radicalmente el manejo del catálogo jerárquico y metadatos de la transmisión de flujos binarios puros, previniendo cuellos de botella en la base de datos de control.
2. **Implementar particionamiento y replicación horizontal:** Fraccionar archivos de gran envergadura en bloques homogéneos (chunks) distribuidos entre nodos de almacenamiento independientes, garantizando un factor de replicación $R \geq 2$.
3. **Ofrecer comunicaciones híbridas optimizadas:** Emplear Remote Procedure Calls (**RPC / gRPC**) para la transferencia sincrónica de datos de alto ancho de banda y Middleware Orientado a Mensajes (**MOM con Colas**) para la absorción de ráfagas, sincronización asíncrona y re-replicación.
4. **Garantizar resiliencia y auto-sanación (self-healing):** Detectar de forma automática nodos caídos mediante *heartbeats* dinámicos y activar la restauración de réplicas sin intervención manual.
5. **Asegurar la privacidad y seguridad perimetral:** Aislar estrictamente los espacios de almacenamiento por usuario, cifrar la información en reposo y en tránsito, y autenticar toda operación mediante tokens criptográficos.

1.3. Supuestos y Limitaciones de Diseño

- **Modelo Jerárquico por Usuario:** Cada usuario registrado posee una raíz de sistema de archivos privada (/home/<username>/). No se contempla el uso compartido de archivos entre diferentes cuentas ni enlaces duros (hard links) multiusuario.
- **Semántica de Archivo:** El sistema no busca conformidad completa con la especificación POSIX clásica, en su lugar, implementa una semántica adaptada a sistemas de archivos distribuidos a gran escala (similar a Google File System o HDFS), priorizando lecturas concurrentes rápidas, reemplazos atómicos y operaciones de adición (append), con escrituras concurrentes coordinadas mediante mecanismos de bloqueo por arrendamiento (leases).
- **Entorno de Red:** La solución será desplegada en una nube privada virtual (VPC). Los nodos de almacenamiento carecen de direcciones IP públicas y solo responden a peticiones provenientes de los controladores autorizados.

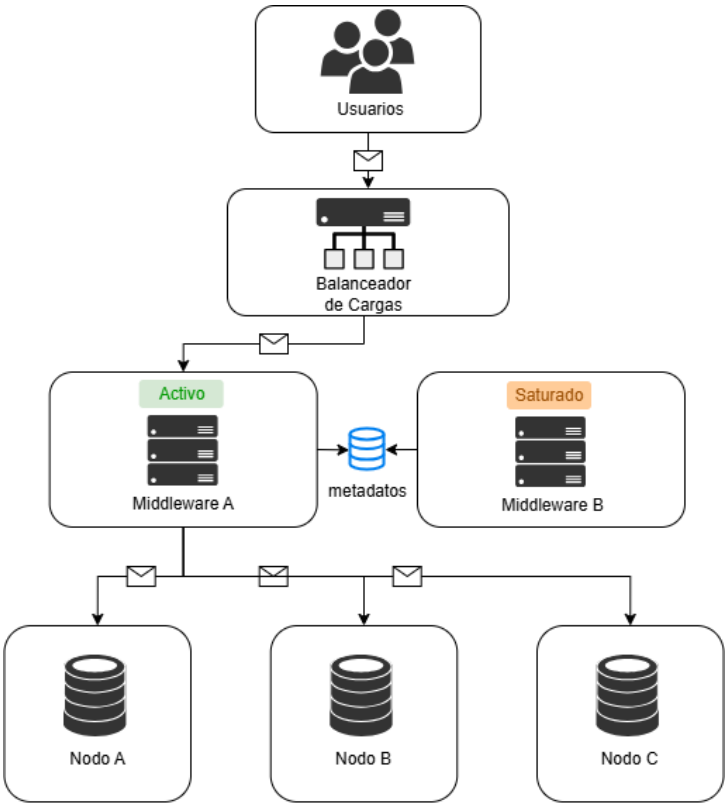
2. Matriz de Cumplimiento de Requisitos

Identificador	Requisito del Sistema	Estrategia de Solución Implementada en DFSha
RF1	Gestión del sistema de archivos jerárquico (ls, cd, mkdir, rmdir, rm). Espacio propio por usuario.	Árbol lógico indexado en la base de datos de metadatos. Cada usuario autenticado opera sobre un espacio aislado /<user_id>/.... Validación estricta en Gateway/Middleware.
RF2	Transferencia de archivos completos (send/put, receive/get).	Particionamiento en streaming binario vía gRPC. Ingesta por bloques, validación de integridad y ensamble transparente al recuperar.
RF3	Primitivas de acceso y control (open, close, read, write, lock, unlock).	Implementación de descriptores de sesión lógica, control de leases distribuidos en metadatos y semántica declarada de escritura atómica y append.
RNF1	Escalabilidad en usuarios, cantidad y tamaño de archivos.	Middlewares sin estado (*stateless*), balanceador de carga elástico, particionamiento en bloques de tamaño fijo (4 MB a 64 MB) y metadatos relacionales optimizados.
RNF2	Alta disponibilidad y redundancia ante fallos.	Factor de replicación R = 2. Conmutación automática a réplicas sanas ante desconexiones. Monitoreo por *heartbeats*.
RNF3	Consistencia de datos y tratamiento de réplicas.	Modelo de consistencia fuerte para metadatos (quórum en transacciones) y consistencia monótona/eventual coordinada para bloques de datos mediante *two-phase commit* simplificado.
RNF4	Particionamiento de archivos entre múltiples nodos.	Algoritmo determinista de selección de nodos con conocimiento de carga (*load-aware placement*). Ningún archivo reside en un único disco.
RNF5	Rendimiento y concurrencia multi-nodo.	I/O en paralelo hacia múltiples trabajadores de almacenamiento; colas MOM (RabbitMQ/Kafka) para descompresión de picos de carga y streaming HTTP/2 binario.
RNF6	Seguridad de datos y comunicaciones.	Autenticación basada en JWT, cifrado en tránsito mediante TLS 1.3 (perímetro) y mTLS (interno S2S), cifrado en reposo con AES-256-GCM y RBAC.
RNF7	Transparencia de acceso y localización.	El cliente no conoce IPs ni puertos de almacenamiento. Toda dirección se resuelve dinámicamente mediante el orquestador de metadatos.

3. Topología y Arquitectura del Sistema

3.1. Visión General de la Arquitectura

El sistema adopta el modelo **Cliente/Servidor con servicio distribuido entre servidores (S2S)**. El clúster interno colabora a través de una red privada para balancear trabajo, replicar bloques y mantener la coherencia.



3.2. Desglose de Componentes y Responsabilidades

Capa de Clientes (CLI, SDK y Frontend Web)

- **Función:** Interfaz expuesta al usuario. Captura comandos jerárquicos (mkdir, ls, cd) y transferencias (put, get).
- **Aislamiento:** No conoce la infraestructura física de nodos, IPs internas ni esquemas de bases de datos. Se comunica exclusivamente con el API Gateway mediante peticiones HTTPS/gRPC autenticadas.
- **Gestión de Sesión:** Se mantiene un JWT en la memoria local del navegador

Capa de Balanceo de Carga y API Gateway (Reverse Proxy)

- **Punto Único de Entrada:** Recibe el tráfico perimetral sobre puerto seguro 443 (TLS 1.3).
- **Inspección de Salud (*Health Checking*):** Mide la saturación de los Middlewares (latencia, uso de CPU, ocupación de colas). Si el Middleware B se satura, desvía el 100% de las nuevas peticiones hacia el Middleware A.
- **Seguridad Perimetral:** Valida firmas criptográficas de tokens JWT, filtra solicitudes maliciosas e impone políticas de limitación de tasa (*Rate Limiting*).

Capa de Coordinación: Clúster de Middlewares (Stateless Controllers)

- **Cerebro del DFSha:** Clúster elástico sin estado. Cualquier nodo middleware puede procesar cualquier petición consultando la base de metadatos.
- **Orquestación de Ingesta:** Fracciona los archivos entrantes en bloques (*chunking*), genera sus sumas de verificación (*checksums* SHA-256) y consulta la base de datos de metadatos para planificar la colocación en nodos con espacio disponible.
- **Coordinación de Lectura:** Obtiene la lista de bloques de un archivo, recupera los chunks concurrentemente desde los nodos de almacenamiento sanos, verifica su integridad y emite el flujo consolidado al cliente.

Capa de Base de Datos de Metadatos

- **Registro de Estado Global:** Base de datos relacional de alta velocidad
- **Exclusión Absoluta de Binarios:** Solo registra esquemas lógicos, rutas de directorio, permisos, listas de chunks y mapeo a nodos de datos. Jamás guarda contenido binario.

Capa de Nodos de Almacenamiento (Storage Workers A, B y C)

- **Persistencia Física Pura:** Servidores de almacenamiento que residen en servidores Linux dentro de la subred privada (VPC).
- **Operaciones Atómicas:** Exponen una interfaz gRPC interna con primitivas de lectura (readChunk), escritura con fsync (writeChunk) y borrado (deleteChunk).

3.4. Modelo Híbrido de Comunicaciones (RPC + MOM)

El sistema combina los modelos de llamadas a procedimientos remotos (RPC) con el modelo de colas de mensajes asíncronas (MOM con QUEUE) tener un sistema robusco y liviano.

Característica	Remote Procedure Call (RPC / gRPC)	Message-Oriented Middleware (MOM con Colas)
Tecnología	gRPC sobre HTTP/2 (Protocol Buffers)	RabbitMQ / Apache Kafka
Naturaleza	Sincrónico, bidireccional, bajo overhead	Asíncrono, persistente, desacoplado temporalmente
Casos de Uso	- Transferencia de bloques binarios - Autenticación y resolución de metadatos en tiempo real - Primitivas de streaming de descarga y subida	- Tareas pesadas de re-replicación tras caída de nodos - Encolamiento de órdenes de purga física de bloques borrados - Amortiguación de ráfagas masivas de solicitudes de escritura
Ventaja Clave	Serialización ultrarrápida y latencia mínima de I/O.	**Resiliencia garantizada:** si un nodo de destino está temporalmente inoperativo, el mensaje espera en cola sin pérdida de datos.

4. Particionamiento, Bloques y Modelo de Metadatos

4.1. Algoritmo de Fraccionamiento (*Chunking*)

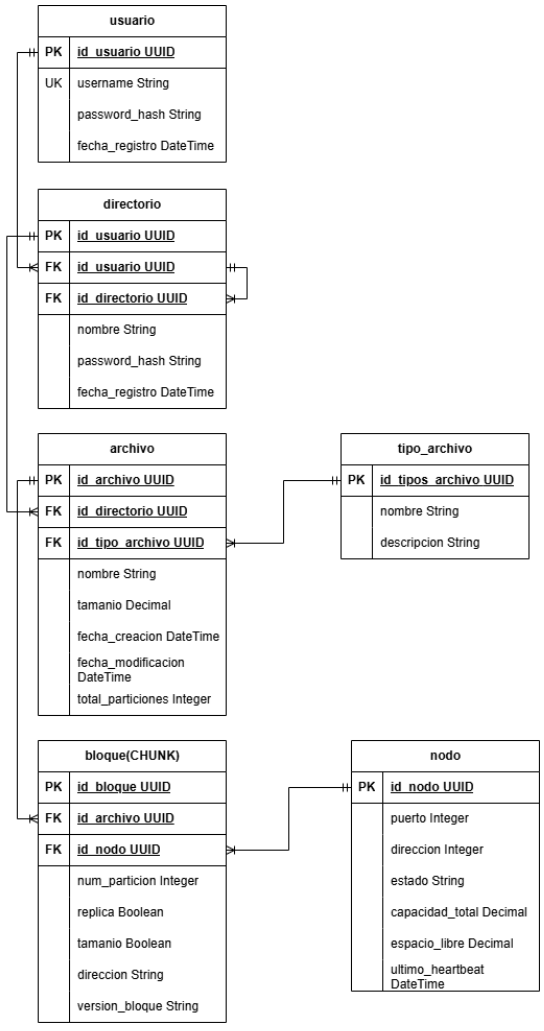
Tamaño Fijo de Bloque: Se define un tamaño estándar de bloque de S = 30MB (configurable entre 4 MB y 64 MB según el perfil de carga).

- Los archivos menores o iguales a 30 MB ocupan un único bloque.
- Un archivo de 100 MB se segmenta en 100/30 = 4 fragmentos: 3 bloques de 30 MB y 1 bloque final de 10 MB.

Identificación Criptográfica: Cada bloque recibe un identificador global único universal (*UUIDv4*) y un hash SHA-256 calculado a partir de su contenido binario.

4.2. Estructura y Esquema de Metadatos

El catálogo de metadatos se organiza en un esquema relacional normalizado:



4.3. Política de Asignación y Replicación ($R \geq 2$)

Factor de Replicación: Se fija un factor por defecto de R = 2 (es el umbral mínimo tolerable).

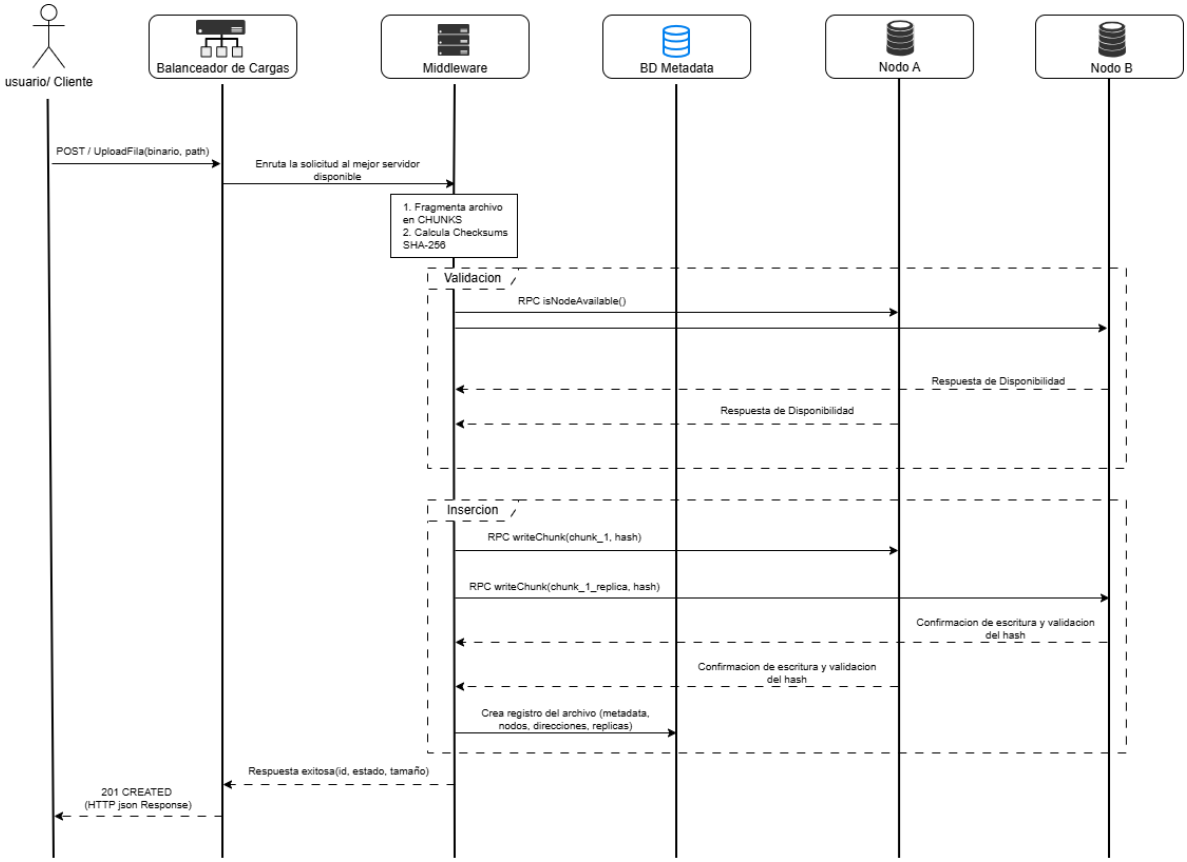
Heurística de Selección de Nodos (*Placement Engine*):

1. **Disponibilidad:** El nodo debe reportar estado ONLINE y haber emitido un heartbeat en los últimos 10 segundos.
2. **Capacidad Relativa:** Se pondera la selección favoreciendo nodos con menor porcentaje de ocupación de disco.
3. **Dispersión:** Dos réplicas del mismo bloque jamás pueden residir en el mismo nodo físico o en el mismo bastidor (*rack-awareness*).

7. Dinámica de Operaciones y Flujos de Datos

7.1. Flujo de Escritura y Carga (put / uploadFile)

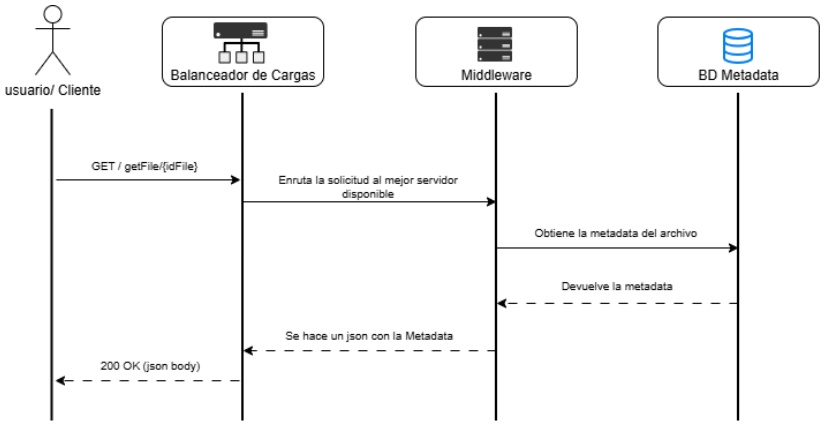
Asegura la fragmentación ordenada, cálculo de hashes y replicación balanceada:



7.2. Flujo de Lectura y Consulta (get y seeMetadata)

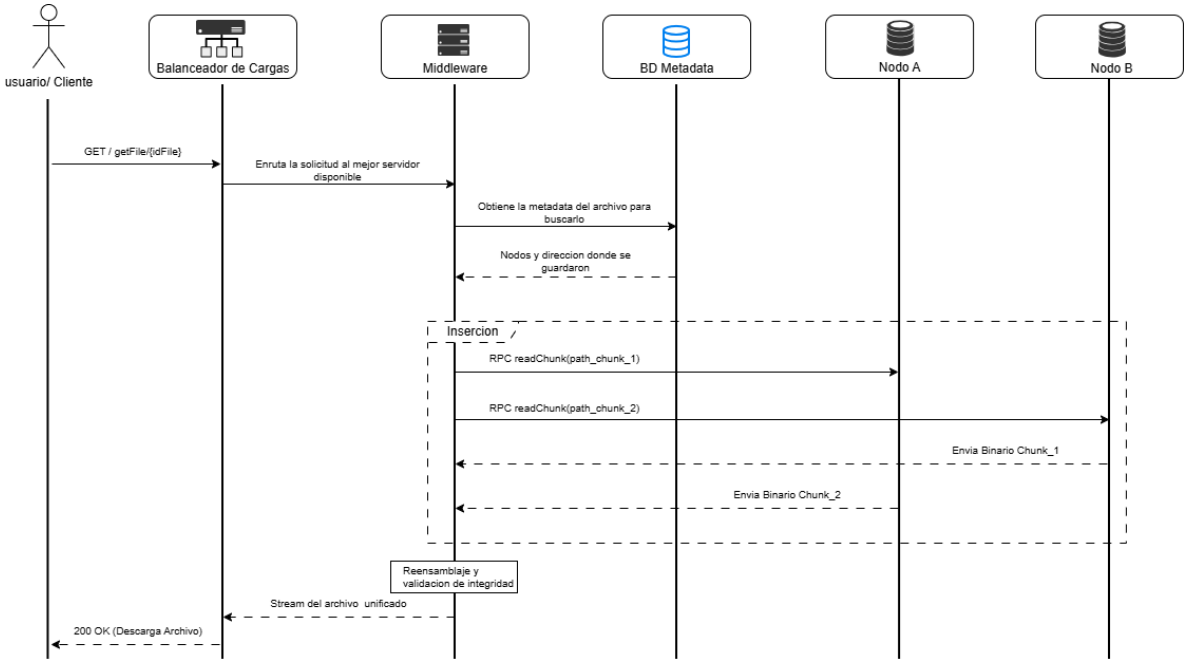
Consulta Rápida de Metadatos (seeMetadata / ls)

Permite conocer atributos, tamaño y verificación sin tocar los discos de datos:



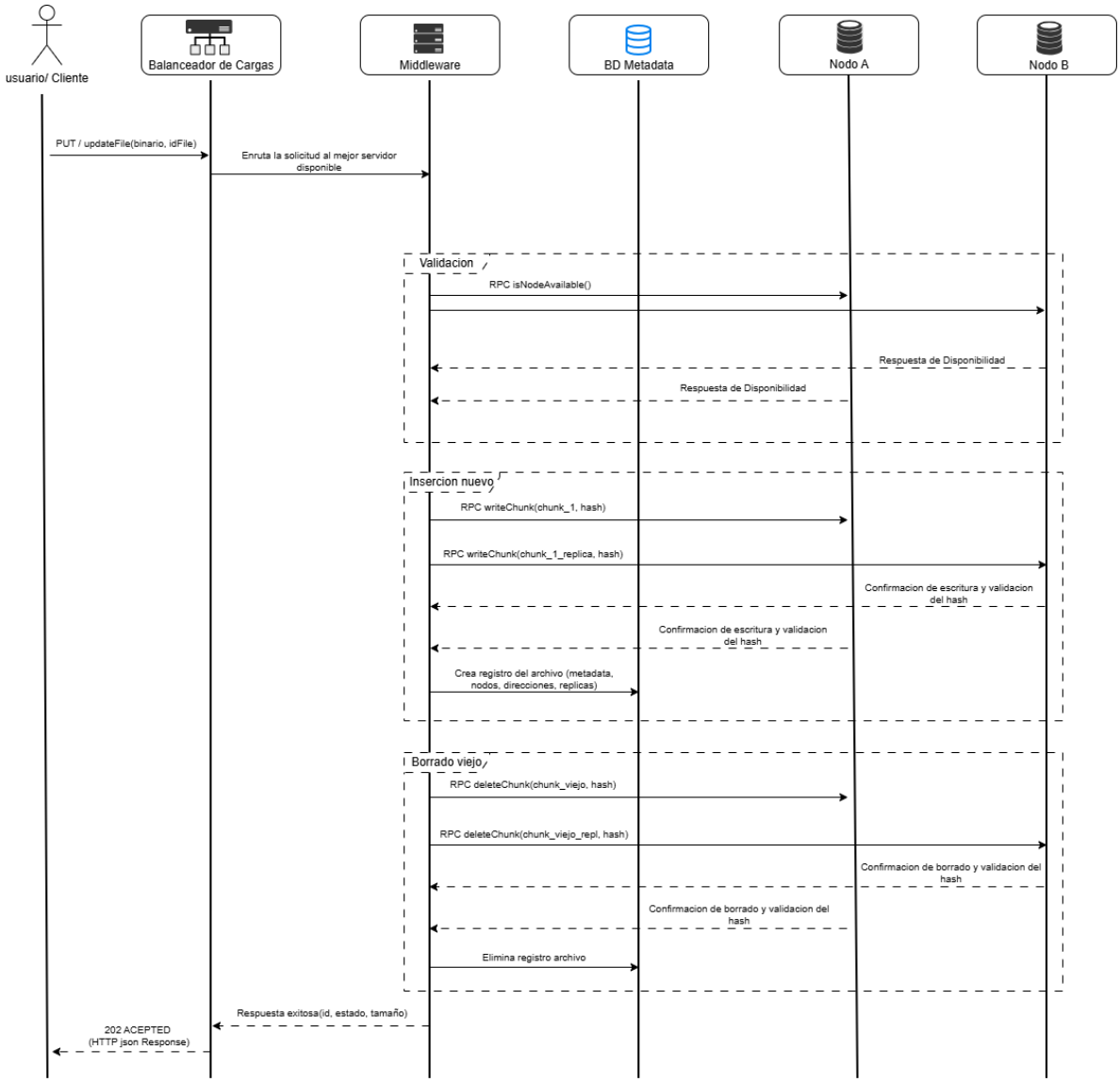
Recuperación de Archivo Completo (getFile / get)

Ensambla y transmite el archivo combinando la lectura paralela de réplicas saludables:



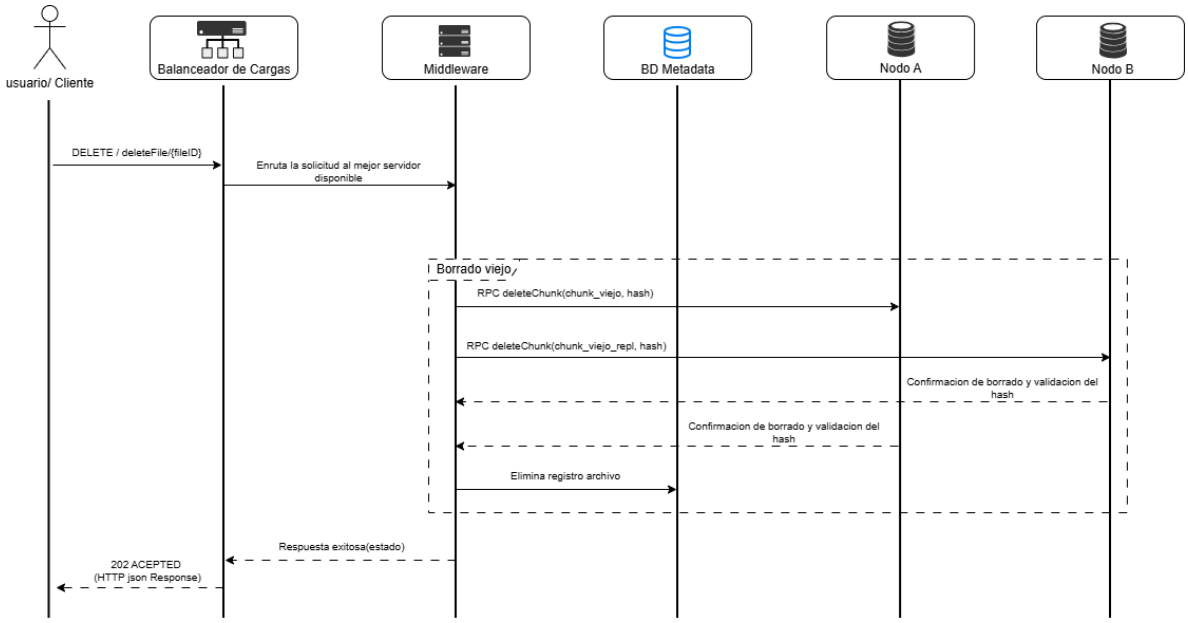
7.3. Flujo de Actualización y Modificación (updateFile)

La modificación de archivos se ejecuta mediante **Copy-On-Write / Versionado Inmutable**:

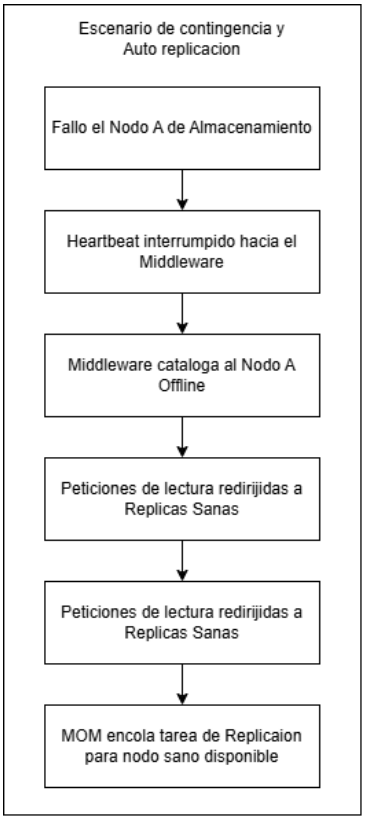


7.4. Flujo de Eliminación (deleteFile / rm)

Garantiza la eliminación inmediata para el usuario y la liberación ordenada de disco físico:



8. Resiliencia, Tolerancia a Fallos y Alta Disponibilidad



8.1. Detección de Caídas Mediante Heartbeats

- Todos los nodos de almacenamiento emiten un pulso gRPC SendHeartbeat cada 10 segundos hacia el clúster de middlewares.
- El pulso contiene porcentaje de uso de CPU, I/O en disco, espacio libre y lista de chunks locales.
- Si transcurren 15 segundos sin recibir el heartbeat de un nodo, el orquestador lo marca automáticamente como OFFLINE o DEGRADED.

8.2. Gestión de Sobrecarga y Saturación de Middlewares

- **Detección Activa:** El Balanceador de Carga monitorea métricas en tiempo real.
- **Mitigación Dinámica:** Si el Middleware B supera el umbral de saturación (por ejemplo, cola de solicitudes en espera > 100 o utilización de CPU > 85%), el balanceador altera los pesos de enrutamiento y conmuta el 100% de nuevas transferencias hacia el Middleware A.
- **Amortiguador MOM:** Peticiones asíncronas de mantenimiento o réplicas se almacenan de forma segura en colas persistentes sin sobrecargar los procesadores del sistema.

8.3. Auto-sanación (*Self-healing*) y Re-replicación

- **Detección de Sub-replicación:** Tras marcarse un nodo como caído, el orquestador identifica qué bloques han quedado con menos réplicas que el factor objetivo ($R < 3$).
- **Plan de Copia en Segundo Plano:** El orquestador inyecta una orden en el MOM. Un nodo saludable que posea una réplica válida (por ejemplo, Worker B) inicia una transmisión directa S2S hacia otro nodo disponible para recrear la réplica faltante.
- **Cero Impacto al Usuario:** Todo el proceso ocurre en segundo plano sin degradar la experiencia ni bloquear las lecturas activas.

9. Seguridad y Aislamiento

9.1. Autenticación y Autorización por Usuario

Identidad Federada / JWT: Al iniciar sesión en el cliente, se emite un JSON Web Token firmado con clave asimétrica RSA-256 o Ed25519.

Aislamiento Estricto de Espacio Lógico:

- Toda ruta relativa ingresada por el usuario es resuelta forzosamente con el prefijo canónico /home/<id_usuario>/.
- Ninguna operación (ls, cd, read, write, rm) puede traspasar las fronteras del árbol del usuario (*chroot lógico*).
- Los intentos de atravesar rutas mediante secuencias ../ son interceptados y bloqueados en el API Gateway.

9.2. Cifrado en Tránsito y Reposo

En Tránsito:

- Todo enlace exterior (Cliente ↔ API Gateway) está protegido con **TLS 1.3** con suites de cifrado modernas (ECDHE-RSA-AES128-GCM-SHA256).
- Las transferencias internas entre Middlewares y Nodos de Almacenamiento utilizan **mTLS (Mutual TLS)** con certificados de corta duración emitidos por una Autoridad Certificadora (CA) interna.

En Reposo:

- Cada bloque persistido en el sistema de archivos del nodo de almacenamiento se cifra mediante **AES-256-GCM**.
- El vector de inicialización (IV) y la etiqueta de autenticación (Tag) se resguardan junto con el fragmento, impidiendo que lecturas no autorizadas del disco físico expongan datos legibles.

9.3. Protección del Tráfico Interno S2S

- Los nodos de almacenamiento carecen de interfaces de red con salida pública.
- Reglas de firewall a nivel de red virtual (Security Groups) restringen el tráfico entrante al puerto gRPC únicamente a las IPs privadas autorizadas del clúster de middlewares.

10. Transparencias del Sistema Distribuido

La arquitectura implementa de manera estricta los principios clásicos de transparencia de sistemas distribuidos:

Dimensión de Transparencia	**Implementación en la Arquitectura DFSha**
Transparencia de Acceso	El cliente opera con comandos estándar idénticos (ls, cd, put, get, open, read, write) independientemente de si los bloques se procesan localmente o se ensamblan desde múltiples servidores remotos.
Transparencia de Ubicación	El usuario accede a los archivos mediante identificadores universales abstractos o rutas lógicas (/home/user/doc.pdf). No necesita conocer direcciones IP, nombres de servidor ni rutas de montaje físico.
Transparencia de Fragmentación	La división del archivo en múltiples bloques homogéneos (*chunks*) y el cálculo de hashes se ejecuta en la capa intermedia; el usuario percibe el archivo como un flujo continuo indivisible.
Transparencia de Replicación	El usuario desconoce cuántas copias de respaldo existen de sus bloques ni en qué nodos de almacenamiento físicos residen para salvaguardar la durabilidad.
Transparencia de Concurrencia	Múltiples clientes pueden consultar y escribir simultáneamente sin percibir inconsistencias, gracias a la sincronización de leases en metadatos y al versionado inmutable (*Copy-On-Write*).
Transparencia de Fallos	Si un nodo de almacenamiento o un middleware colapsa, el tráfico es redirigido en milisegundos hacia réplicas sanas o controladores activos sin arrojar errores al usuario final.
Transparencia de Escalabilidad	Se pueden añadir dinámicamente nuevos trabajadores de almacenamiento o middlewares al clúster sin necesidad de reiniciar el servicio ni modificar la configuración del cliente.

11. Escenarios Obligatorios de Razonamiento

11.1. Pregunta: *Dos procesos intentan escribir el mismo bloque de un archivo simultáneamente: ¿qué sucede y por qué?*

Resolución en DFSha:

1. Cuando el **Proceso 1** y el **Proceso 2** solicitan escribir concurrentemente el mismo bloque, ambas solicitudes llegan al clúster de middlewares y compiten atómicamente por el *lock*.
2. La base de datos de metadatos procesa una transacción serializable: el Proceso 1 obtiene el *lock* exclusivo para el bloque y recibe un token de arrendamiento temporal con expiración.
3. La solicitud del Proceso 2 es interceptada por el middleware, retornando de inmediato un código de conflicto 409 Conflict (o manteniéndola en espera activa si se solicitó con parámetro de espera diferida en la cola MOM).
4. Adicionalmente, gracias a la estrategia de **Copy-On-Write (COW)**, el Proceso 1 no destruye el contenido original in-situ, sino que genera una nueva versión del bloque con los cambios aplicados. Solo cuando las réplicas confirman la escritura con éxito, se actualiza el puntero de versión en los metadatos.
5. **Resultado:** Se previene de forma absoluta la intercalación destructiva de bytes (*corrupted interleaving*) y se asegura consistencia serializable.

11.2. Pregunta: *Una réplica está desconectada durante una escritura: ¿qué observan los lectores y cómo se recupera?*

Resolución en DFSha:

1. **Comportamiento durante la Escritura:** Al tenerse un factor de replicación R = 2, el sistema va a asegurar que haya al menos otra copia disponible en el caso que falle un nodo. Por ejemplo, Si el Nodo A y el Nodo B están activos pero el Nodo C está desconectado, el middleware detecta por medio de los *heartbeat*s que el Nodo C no está disponible, y escribe concurrentemente en A y B, cumpliendo satisfactoriamente la escritura y consolidándola como exitosa en los metadatos.
2. **Qué Observan los Lectores:**
    a. Cualquier lector que consulte el archivo a través del middleware será enrutado exclusivamente a los nodos que poseen la versión actualizada (Nodo A o Nodo B).
    b. Dado que el catálogo de metadatos marca la réplica del Nodo C con el estado DESACTUALIZADA o OFFLINE, ningún lector será dirigido a dicho nodo. Por tanto, los lectores observan en todo momento una **visión monótona y consistente** de los datos.
3. **Cómo se Recupera:**
    a. Cuando el Nodo C vuelve a conectarse y emite su latido (*heartbeat*), el orquestador compara el número de versión local de sus bloques con el catálogo central.
    b. Al detectar que el bloque en C tiene una versión obsoleta, el middleware publica una orden de sincronización en el MOM.
    c. El Nodo C solicita la copia actualizada desde el Nodo A o B, valida el checksum SHA-256, actualiza su disco local y notifica a los metadatos para pasar su estado a SINCRONIZADA.

11.3. Pregunta: *Un nodo falla después de recibir parte de un bloque: ¿cómo se evita publicar un archivo incompleto?*

Resolución en DFSha:

1. **Aislamiento en Espacio de Trabajo Temporal (*Staging Buffer*):** Los nodos de almacenamiento jamás escriben directamente sobre el archivo final visible. Los bytes recibidos por streaming gRPC se escriben en un archivo temporal con extensión .part o en un directorio de *staging* asignado a la transacción.
2. **Validación Criptográfica Estricta:** El nodo solo renombra el archivo temporal a su identificador definitivo tras recibir la señal de fin de stream (EOF) y constatar que el tamaño en bytes coincide con el esperado y que el hash SHA-256 recalculado es idéntico al emitido en los metadatos.
3. **Confirmación en Dos Fases Simplificada:**
   - Si el nodo colapsa mientras transfería el 40% del bloque, la conexión gRPC se interrumpe abruptamente.
   - El middleware detecta el fallo por timeout en el socket y aborta la transacción para dicho nodo.
   - Al no alcanzarse el quórum de confirmaciones, el middleware **no registra el nuevo bloque en la base de metadatos**.
   - Para cualquier cliente o lector del sistema, el archivo incompleto sencillamente **no existe**, preservando la propiedad de atomicidad (*todo o nada*).
4. **Purga de Residuos:** Al reiniciarse el nodo caído, su proceso demonio ejecuta una rutina de limpieza inicial que elimina cualquier archivo parcial huérfano (*.part) con una antigüedad superior a 15 minutos.

11.4. Pregunta: *El cliente desconoce la ubicación física de los bloques: ¿cómo los descubre dinámicamente?*

Resolución en DFSha:

1. **Desacoplamiento Absoluto:** El cliente no almacena tablas de enrutamiento estáticas ni IPs cableadas (*hardcoded*) de los nodos de almacenamiento; solo conoce la URL o dirección virtual del **API Gateway / Balanceador de Cargas**.
2. **Protocolo de Resolución Dinámica:**
    - El cliente invoca la operación indicando únicamente la ruta lógica (ej. GET /api/v1/files/download?path=/home/squiceno/reporte.pdf).
    - El Gateway enruta la petición al middleware disponible.
    - El middleware consulta el catálogo de metadatos, el cual resuelve la jerarquía:

$$\text{Ruta Lógica} \rightarrow \text{ID Archivo} \longrightarrow [\text{Chunk}_0, \text{Chunk}_1, \dots, \text{Chunk}_n] \longrightarrow \left[\text{Nodos con réplica activa}\right]$$

3. **Entrega Transparente Mediante Proxy:**
    - En el diseño, el middleware actúa como orquestador transparente: es él quien contacta a los nodos de almacenamiento en la red privada, obtiene los fragmentos, verifica su integridad y transmite el flujo ya ensamblado al cliente en un único stream HTTP/2 o gRPC.
    - El cliente recibe el archivo completo de forma inmediata sin haberse comunicado jamás directamente con ninguno de los servidores de almacenamiento físicos.
4. **Ventaja:** Permite mover bloques entre nodos, retirar servidores por mantenimiento o rebalancear discos sin requerir ninguna actualización en las aplicaciones cliente.

12. Plan de Pruebas, Métricas y Evaluación Experimental

12.1. Métricas Clave de Desempeño

Para validar empíricamente la arquitectura en la Fase II, se instrumentarán las siguientes métricas:

Métrica	Descripción	Unidad de Medida	Herramienta de Medición
Throughput de Escritura	Tasa sostenida de ingesta de datos en el sistema.	MB/s	Medición en Gateway y Nodos de Datos
Throughput de Lectura	Velocidad de recuperación y ensamble de archivos.	MB/s	Medición en Cliente y Middleware
Latencia de Operaciones de Control	Tiempo de respuesta para comandos jerárquicos (ls, mkdir, cd).	Milisegundos	Prometheus + Grafana
Tiempo de Recuperación ante Fallos (MTTR)	Tiempo transcurrido entre la caída de un nodo y la restauración del factor de replicación $R = 3$.	Segundos	Logs de eventos MOM y Metadatos
Sobrecarga de Red por Replicación	Volumen de bytes duplicados en la red interna para mantener redundancia.	Ratio de amplificación de red	Métricas de interfaz Linux (ifconfig/nload)

12.2. Batería de Experimentos Propuestos

1. **Prueba de Escalabilidad de Tamaño de Archivo:**
    - Evaluar el comportamiento del sistema almacenando archivos de **10 MB, 100 MB, 1 GB y 5 GB**.
    - **Objetivo:** Comprobar que el uso de memoria en el Middleware permanece constante e independiente del tamaño del archivo gracias al streaming de chunks de 16 MB.
2. **Prueba de Concurrencia Masiva:**
    - Simular de 10 a 100 clientes concurrentes leyendo y escribiendo archivos simultáneamente mediante herramientas de carga (ej. k6 o Locust).
    - **Objetivo:** Medir la degradación de latencia y validar el algoritmo de balanceo adaptativo cuando el Middleware B se satura.

Nota Academica

Alcance del Documento y Consideraciones para la Fase II

El diseño presentado en este informe busca explorar los máximos estándares de tolerancia a fallos, consistencia y seguridad para un sistema distribuido de gran escala.

Por ende, este documento debe entenderse como una **propuesta de diseño arquitectónico (Fase I)** y no como una promesa definitiva de desarrollo en su totalidad para la entrega final. Servirá como base para que el equipo evalúe, debata y defina la arquitectura ejecutable de la Fase II, ajustando compromisos técnicos (*trade-offs*) y priorizando componentes según la viabilidad de implementación.